

FULL STACK WEB DEVELOPMENT

USE CASE – ASSIGNMENT

Subject Code: 21CSE354T

STUDENT NAME	PARTH BHAVINBHAI SAVALIYA
REGISTER NUMBER	RA2411003050001
PROGRAM / SECTION	B. TECH CSE - A
YEAR / SEMESTER	III / V
ASSIGNMENT / USE CASE NO.	22
DATE OF SUBMISSION	31/08/2026

COURSE	Full Stack Web Development
FACULTY COORDINATOR	Dr. Hariharan
ACADEMIC YEAR	2026 – 2027

“Design • Develop • Deploy”

Table of Contents

- 1. Executive Summary..... 2
- 2. Introduction..... 2
- 3. Problem Statement and Existing Situation..... 2
- 4. Objectives and Requirements..... 3
- 5. System Architecture and Data Flow..... 5
- 6. Technology Stack and Design Decisions..... 6
- 7. Features and Functional Modules..... 7
- 8. Implementation and DOM Logic..... 8
- 9. User Interface and Screenshots..... 9
- 10. Testing and Results..... 11
- 11. Limitations, Security and Future Scope..... 12
- 12. Deployment and Maintenance..... 12
- 13. Conclusion..... 13

1. Executive Summary

This report documents the design, development, and evaluation of the Image Upload & Preview Web Application, a frontend-only project built to let a user select an image from their device and see an immediate, in-browser preview of that image before choosing to submit it. The application is implemented entirely with HTML5, CSS3, and vanilla JavaScript, using the browser's native FileReader API to read the selected file and render it as a preview without any server-side component.

The interface follows a modern dark and purple gradient visual style with a glassmorphism-effect central card, a dashed drag-style upload zone, a dedicated preview panel, a click-to-enlarge preview state, and a temporary success notification shown after a valid submission. The project was version-controlled with Git, hosted on GitHub, and deployed as a static site on Vercel.

This report presents the problem the project addresses, the objectives and requirements that guided its development, the client-side architecture and data flow, the technology choices, a walkthrough of each functional module supported by screenshots of the working application, the testing carried out, the current limitations, and the future scope for extending the project into a full-stack solution.

2. Introduction

Selecting a file for upload is one of the most common interactions on the modern web, yet the default browser file input offers almost no visual feedback: it shows only a filename, with no indication of what the selected image actually looks like. The Image Upload & Preview Web Application was built to address this gap by giving the user an instant, in-browser preview of the exact image they have chosen, before any submission takes place.

Immediate visual feedback of this kind improves usability in several concrete ways: it lets the user confirm they have selected the correct file, it reduces the chance of submitting the wrong image, and it makes the interaction feel responsive rather than opaque. Because the preview and validation logic run entirely in the browser using the FileReader API and DOM manipulation, the feedback appears instantly, with no network request and no waiting on a server.

The project is deliberately implemented on the frontend only. This keeps the scope focused on core client-side web development skills — HTML structure, CSS-based visual design, and JavaScript-driven interactivity — and allows the application to be demonstrated and deployed without any backend infrastructure, database, or authentication layer.

The technologies used are HTML5 for page structure, CSS3 for the dark/purple glassmorphism visual design and responsive layout, and JavaScript for DOM manipulation and use of the FileReader API. Git and GitHub were used for version control and source hosting, and the finished build was deployed to Vercel as a static site.

2.1 Learning Outcomes

Building this project reinforced practical skills in DOM element selection and event handling, asynchronous file reading with the FileReader API, conditional class toggling for UI state changes, form-style validation without a real form submission, and the use of `setTimeout()` to manage temporary UI states such as a success notification. It also gave hands-on experience with a modern CSS visual language — gradients, glassmorphism, and responsive layout — and with a complete Git-to-deployment workflow using GitHub and Vercel.

3. Problem Statement and Existing Situation

3.1 Existing Situation

A basic HTML file input (`<input type="file">`) satisfies the minimum requirement of letting a user pick a file, but it has clear usability limitations for an image-centric task:

- It displays only a filename or a generic "No file chosen" label, with no visual representation of the image itself.
- Users receive no immediate confirmation that the correct image was selected before proceeding further.
- The default control is difficult to style, so it looks inconsistent with a modern, branded interface.
- There is no built-in way to inspect the image more closely (for example, by enlarging it) before submitting.

Taken together, these limitations can leave users uncertain about whether their selection was registered correctly, which is a common source of confusion and accidental submission of the wrong file.

3.2 Proposed Solution

The Image Upload & Preview Web Application replaces the default file input with a styled, clickable upload area that triggers a hidden file input, and immediately renders the selected image in a dedicated preview panel using JavaScript and the FileReader API. The selected filename is displayed alongside the preview, the preview can be clicked to enlarge it, and a Submit button is validated so that submission is blocked until an image has been selected. A temporary green success notification confirms a valid submission.

3.3 Functional Requirements

Requirement	Description	Status
Custom upload trigger	A styled upload area that opens the file picker via a hidden file input.	Implemented
Instant image preview	Selected image is displayed in a preview panel without page reload.	Implemented
Filename display	Name of the selected file is shown beneath the preview.	Implemented
Enlarge preview	Clicking the preview enlarges the image for closer inspection.	Implemented
Submission validation	Submit button checks that an image has been selected first.	Implemented
Success feedback	A temporary success notification confirms submission.	Implemented
Responsive layout	Interface adapts to different screen sizes.	Implemented
Backend storage	Persisting the image on a server or database.	Out of scope

4. Objectives and Requirements

The project was guided by a small set of clear objectives spanning usability, technical implementation, visual design, responsiveness, client-side processing, and deployment.

4.1 Objectives

Usability objectives: give the user instant confirmation of the image they have selected, and prevent submission when no image has been chosen.

Technical objectives: implement the entire upload-and-preview flow using DOM manipulation and the FileReader API, without relying on any backend service.

UI objectives: present the upload and preview flow inside a visually modern interface using a dark/purple gradient theme and a glassmorphism-style card.

Responsive design objectives: ensure the upload card, preview panel, and buttons remain usable and well-proportioned across desktop and mobile screen sizes.

Client-side processing objectives: read and render the image entirely in the browser using `readAsDataURL()`, with no image data leaving the device.

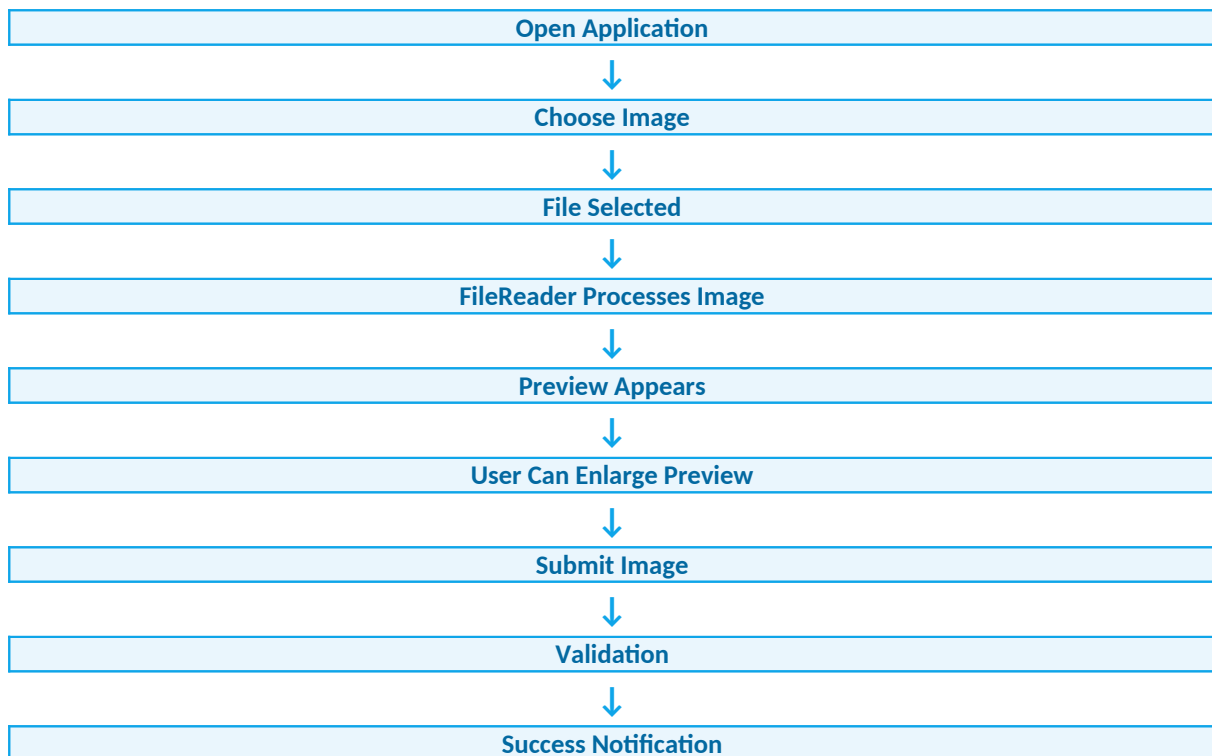
Deployment objectives: version the project with Git, host the source on GitHub, and deploy a live, publicly accessible build on Vercel.

4.2 Requirement Implementation Matrix

Category	Requirement	Implementation Approach
Usability	Confirm selected image before submission	Preview rendered instantly via <code>FileReader</code> on the change event
Technical	No backend dependency	All logic runs client-side in JavaScript; no server calls made
UI	Modern, branded visual identity	Dark/purple gradient background with a glassmorphism card
Responsive	Usable on mobile and desktop	CSS responsive sizing and flexible layout containers
Client-side	Read image data without uploading it	<code>FileReader.readAsDataURL()</code> converts file to a data URL
Deployment	Public, live access to the project	Git → GitHub repository → Vercel static deployment

4.3 User Journey

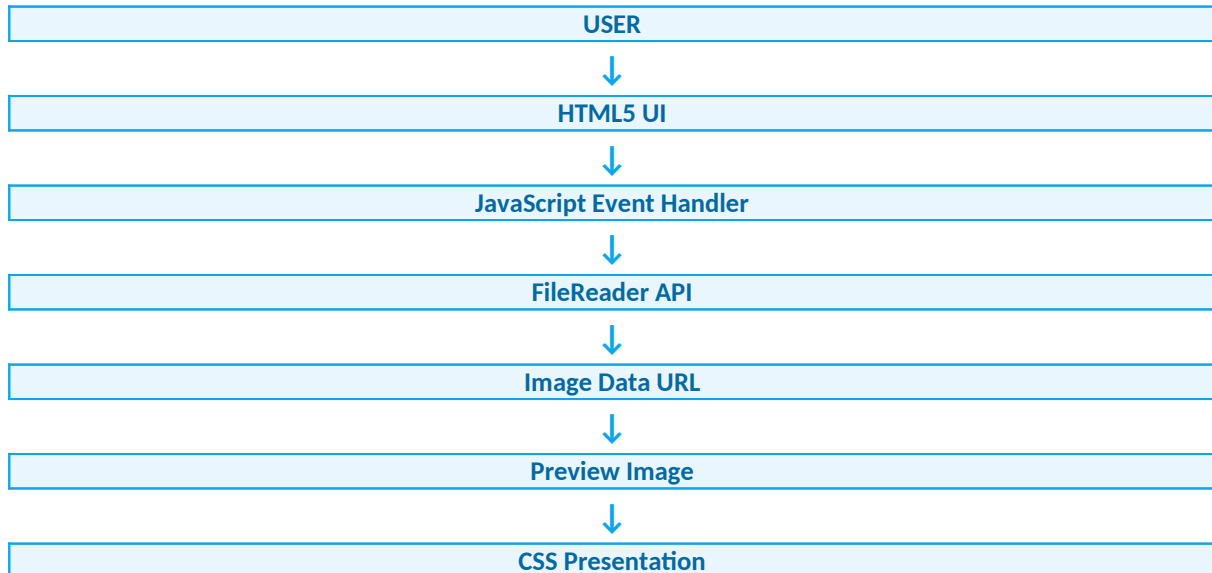
The end-to-end user journey through the application follows a predictable, linear flow:



5. System Architecture and Data Flow

The Image Upload & Preview Web Application uses a purely client-side architecture. There is no application server, API layer, or database — a single HTML document, its stylesheet, and a JavaScript file are loaded by the browser, and every interaction, from file selection to preview rendering to the success notification, is handled entirely within the browser's JavaScript engine.

5.1 Architecture Flow



5.2 How Data Moves Through the Application

When the user clicks the styled upload area, a hidden file input is triggered and the operating system's native file picker opens. Once the user selects an image, the input's change event fires, and JavaScript retrieves the corresponding File object from the input's files list. A FileReader instance is created and its `readAsDataURL()` method is called on that File object, which asynchronously reads the binary image data and encodes it as a base64 data URL.

When the FileReader's onload event fires, the resulting data URL is assigned directly to the `src` attribute of the preview `<img>` element via DOM manipulation, which causes the browser to render the selected image immediately. At the same time, the placeholder text ("Your image will appear here") is hidden, and the selected file's name is written into a dedicated filename element. From this point, CSS governs how the preview panel, image, and surrounding card are presented, including the enlarged state triggered when the user clicks the preview.

5.3 Data Flow Summary

Stage	Trigger	Component Involved	Resulting State
File selection	User clicks upload area, chooses a file	Hidden file input	change event fires
File read	change event handler runs	<code>FileReader.readAsDataURL()</code>	Image encoded as data URL
Preview render	FileReader onload event	Preview <code></code> element	Image displayed, placeholder hidden
Filename display	Same onload/change handler	Filename text element	Selected filename shown

Stage	Trigger	Component Involved	Resulting State
Zoom	User clicks the preview image	CSS class toggle	Enlarged preview state
Submission	User clicks Submit Image	Validation logic	Success notification or blocked submission

6. Technology Stack and Design Decisions

Technology	Role	Reason for Selection
HTML5	Page structure	Provides the semantic upload area, preview container, and button markup
CSS3	Visual styling	Implements the dark/purple gradient theme, glassmorphism card, and responsive layout
JavaScript	Application logic	Handles events, DOM updates, validation, and the success notification
DOM	Interface manipulation	Reads and updates elements such as the preview image, filename text, and CSS classes
FileReader API	Client-side file reading	Converts the selected image file into a displayable data URL without a server
Git	Version control	Tracks changes to the project source over time
GitHub	Source hosting	Hosts the public repository and enables collaboration and history tracking
Vercel	Deployment	Serves the static build as a live, publicly accessible website

6.1 Visual Design System

The interface uses a dark background with a purple-to-blue gradient, giving the application a modern, premium feel. The central upload card uses a glassmorphism effect — a semi-transparent, blurred panel with rounded corners and a soft border — which sits on top of the gradient background and visually separates the interactive controls from the page backdrop.

The upload zone itself uses a dashed border to signal that it is a drop-style interactive region, and the preview panel is a clearly bounded container beneath it so the image preview and the upload control never compete visually. The Submit Image button uses the same purple gradient as an accent, reinforcing a consistent colour identity throughout the interface.

6.2 Responsive Design and Interaction Design

CSS-based responsive sizing allows the central card, upload area, and preview panel to resize gracefully across desktop and mobile viewport widths, keeping the layout centred and legible. Hover and click interactions — such as highlighting the upload area on hover and enlarging the preview on click — are implemented with CSS transitions so state changes feel smooth rather than abrupt.

6.3 Why Vanilla JavaScript

Vanilla JavaScript was used instead of a frontend framework because the application's logic is self-contained — a handful of event listeners, one FileReader call, and a small number of DOM updates. Introducing a framework would add build tooling and dependency overhead without a proportional

benefit, whereas plain JavaScript keeps the project lightweight, easy to audit, and simple to deploy as a static site with no build step.

7. Features and Functional Modules

7.1 Upload Module

Purpose: Let the user choose an image from their device through a clearly styled entry point.

How it works: A dashed, clickable upload area sits above the preview panel. Clicking it programmatically triggers a hidden `<input type="file">` element, which opens the operating system's native file picker.

User benefit: The user gets an on-brand, clearly labelled control instead of the plain, hard-to-style default file input.

Technology involved: HTML5 file input, CSS styling, JavaScript click delegation.

7.2 Image Preview Module

Purpose: Show the selected image immediately, without requiring submission first.

How it works: On the input's change event, JavaScript reads the chosen File object with `FileReader.readAsDataURL()` and, once loaded, sets the resulting data URL as the src of the preview `<img>` element.

User benefit: The user can visually confirm the correct image was chosen before going any further.

Technology involved: FileReader API, DOM manipulation.

7.3 Filename Display Module

Purpose: Show the name of the selected file alongside the visual preview.

How it works: The selected File object's name property is read and written into a dedicated text element beneath the preview panel, replacing the default "No image selected" text.

User benefit: Gives an additional, text-based confirmation of the exact file selected.

Technology involved: JavaScript, DOM text content update.

7.4 Preview Zoom Module

Purpose: Let the user inspect the selected image more closely.

How it works: Clicking the preview image toggles a CSS class on the preview container that enlarges its size and adjusts positioning; clicking again (or clicking elsewhere) removes the class to restore the normal preview size.

User benefit: Useful for checking image details, such as focus or cropping, before submitting.

Technology involved: JavaScript class toggling, CSS transitions.

7.5 Submit Module

Purpose: Allow the user to confirm and finalise their image selection.

How it works: The Submit Image button triggers a click handler that first checks whether an image has been selected, then, if valid, displays the success notification.

User benefit: Gives the interaction a clear, explicit completion step, rather than treating selection alone as submission.

Technology involved: JavaScript event handling.

7.6 Validation Module

Purpose: Prevent submission when no image has been selected.

How it works: The submit handler checks whether the preview's src (or a stored reference to the selected file) is set. If not, submission is blocked and no success notification is shown.

User benefit: Avoids a false-positive confirmation when the user has not actually chosen a file.

Technology involved: JavaScript conditional logic.

7.7 Success Notification Module

Purpose: Confirm to the user that a valid submission has been registered.

How it works: On a valid submission, a green notification element is shown (for example, by toggling a visibility class), and `setTimeout()` is used to automatically hide it again after a short delay.

User benefit: Provides clear, temporary feedback without requiring the user to dismiss anything manually.

Technology involved: JavaScript, `setTimeout()`, CSS transitions.

7.8 Responsive Design Module

Purpose: Keep the interface usable and visually balanced across screen sizes.

How it works: CSS flexible sizing and relative units allow the central card, upload area, and preview panel to adapt their width and spacing depending on the viewport.

User benefit: The application remains comfortable to use on both desktop and mobile devices.

Technology involved: CSS3 responsive layout.

7.9 UI State Summary

State	Condition	Visible Behaviour
Initial	No image selected	Placeholder text "Your image will appear here" shown; "No image selected" label
Selected	Image chosen via file picker	Preview image and filename displayed
Zoomed	Preview image clicked	Preview enlarges to a closer view
Blocked	Submit clicked with no image	Submission prevented; no success notification shown
Submitted	Submit clicked with a valid image	Green success notification appears, then auto-hides

8. Implementation and DOM Logic

This section explains, conceptually, how the JavaScript logic behind the upload-and-preview flow is structured, focusing on the reasoning behind each step rather than the raw code itself.

8.1 Selecting DOM Elements

At the top of the script, references to the key interface elements — the hidden file input, the upload area, the preview image, the filename text element, the preview container, the submit button, and the success notification — are retrieved once using methods such as `getElementById` or `querySelector`, and stored in variables for repeated use.

Clicking the styled upload area calls the `.click()` method on the hidden file input, which opens the operating system's native file picker without exposing the default, unstyled control to the user. This is the mechanism that lets a custom, branded upload zone stand in for the plain browser control while still relying on it under the hood.

8.2 Detecting File Selection and Reading the Image

A change event listener on the file input fires whenever the user selects a file. Inside the handler, the selected File object is retrieved from the input's `files` property, a new `FileReader` instance is created, and its `readAsDataURL()` method is called to begin reading the file asynchronously.

The reader's `onload` event fires once the file has been fully read. At that point, the event's `result` property contains the base64-encoded data URL of the image, which is assigned directly to the preview image's `src` attribute, causing the browser to render it immediately. In the same step, the placeholder text ("Your image will appear here") is hidden and the filename text element is updated to show the selected file's name.

8.3 Toggling the Enlarged Preview

Clicking the preview image toggles a CSS class, such as "enlarged", on the preview container. CSS rules associated with that class increase the image's size and adjust its positioning, while a transition property ensures the change animates smoothly rather than snapping instantly. Clicking the preview again removes the class and restores the normal preview size.

8.4 Submit Handling and Validation

The submit button's click handler first checks whether an image has actually been selected — typically by checking whether the preview image's `src` has been set to a real data URL rather than being empty. If no image has been selected, the handler exits early and no success notification is shown. If an image has been selected, the success notification is revealed.

A `setTimeout()` call schedules the removal of the notification's visibility class after a fixed delay, so the success notification automatically disappears without requiring the user to close it manually. Because the project is frontend-only, this success state represents confirmation of a valid, client-side interaction rather than a real network upload.

9. User Interface and Screenshots

The following screenshots were captured directly from the deployed application and illustrate each major state of the upload-and-preview flow.

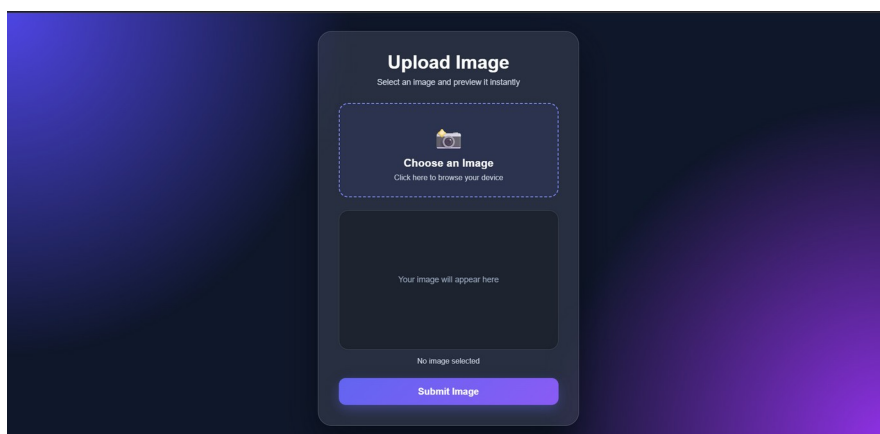


Figure 1 — Initial state of the application, showing the styled upload area and the empty preview panel before any image has been selected.

Figure 1 shows the application on first load: the glassmorphism card, the dashed "Choose an Image" upload area, the placeholder text "Your image will appear here" in the preview panel, and the "No image selected" label above the Submit Image button.

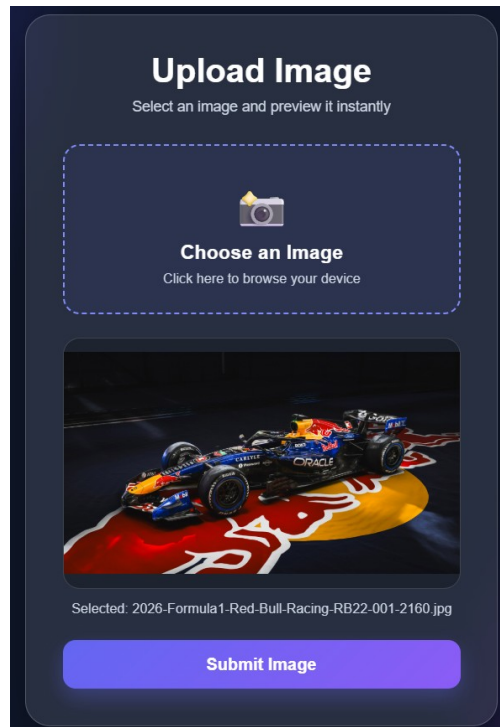


Figure 2 — An image has been selected; the preview panel displays the chosen image and the filename is shown beneath it.

Figure 2 demonstrates the core preview functionality: after choosing an image through the file picker, the FileReader-driven preview updates instantly, the placeholder text is replaced by the actual image, and the filename "2026-Formula1-Red-Bull-Racing-RB22-001-2160.jpg" is displayed above the Submit Image button.



Figure 3 — The preview image enlarged after being clicked, allowing closer inspection of the selected image.

Figure 3 shows the enlarged preview state, triggered by clicking the preview image. The CSS-driven zoom presents the image at a larger size against a darkened backdrop, letting the user inspect details before submitting.

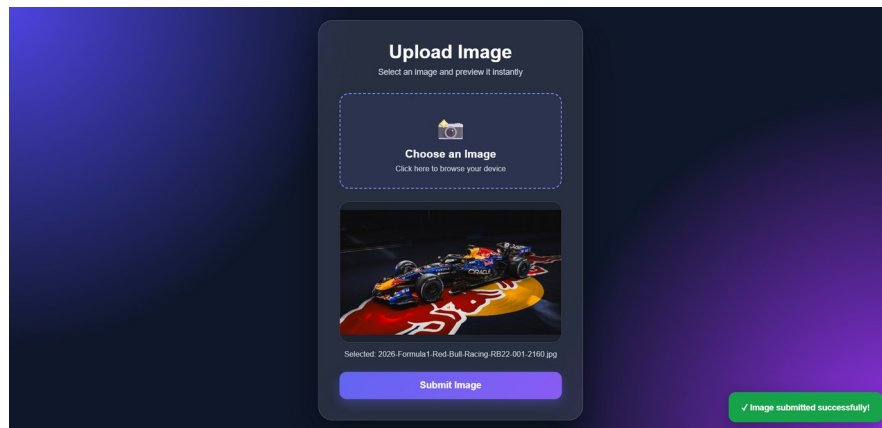


Figure 4 — The green success notification shown after a valid image submission.

Figure 4 shows the state after clicking Submit Image with a valid image selected: a green "Image submitted successfully!" notification appears in the corner of the screen, confirming the client-side validation passed. This notification is automatically hidden again after a short delay using `setTimeout()`.

10. Testing and Results

The application was manually tested against the scenarios below to confirm that each functional module behaves as expected.

Test ID	Action	Expected Result	Actual Result	Status
T01	Open application	Upload card and empty preview panel are displayed	As expected	Pass
T02	Click upload area	Native file picker opens	As expected	Pass
T03	Select an image file	File is accepted and change event fires	As expected	Pass
T04	Check preview after selection	Selected image is rendered in the preview panel	As expected	Pass
T05	Check filename after selection	Selected filename is displayed beneath the preview	As expected	Pass
T06	Click the preview to enlarge	Preview enlarges via CSS class toggle	As expected	Pass
T07	Click again to restore preview	Preview returns to its normal size	As expected	Pass
T08	Click Submit with no image selected	Submission is blocked; no success notification shown	As expected	Pass
T09	Click Submit with a valid image selected	Green success notification appears, then auto-hides	As expected	Pass
T10	Refresh the page	Application resets to its initial, empty state	As expected	Pass
T11	Resize the browser / test on mobile width	Layout reflows and remains usable	As expected	Pass

All eleven test cases produced the expected result. The validation logic correctly blocks submission when no image has been selected (T08), and the preview, filename display, and zoom behaviours (T04–T07)

responded instantly with no perceptible delay, confirming that the client-side FileReader-based approach is well suited to this use case. The responsive layout also held up under different viewport widths (T11), and because the project is entirely static, refreshing the page (T10) simply reloads the initial state, with no data persisted, consistent with the project's frontend-only design.

11. Limitations, Security and Future Scope

11.1 Current Limitations

As a frontend-only application, the project has a clearly bounded set of limitations that should be understood before it is treated as anything more than a client-side demonstration:

- No backend: there is no server component processing or receiving the selected image.
- No database: no record of uploaded images is stored anywhere.
- No permanent storage: the image exists only in the browser's memory for the duration of the page session and is lost on refresh or navigation.
- No authentication: the application does not identify or distinguish between users.
- No cloud image storage: images are never transmitted to or stored on any external service.

The green success notification shown after submission represents client-side UI feedback only — confirmation that the validation logic accepted a selected image — and does not indicate that the image has been uploaded to, or stored on, any server.

11.2 Production Security Considerations

If this project were extended with a real backend, several security measures would need to be introduced before accepting uploads in production: server-side validation of file type and size (rather than trusting client-side checks alone), MIME-type verification, virus/malware scanning of uploaded files, authenticated upload endpoints, and rate limiting to prevent abuse. None of these are currently applicable because no file is ever transmitted off the user's device.

11.3 Future Scope

- Backend API to receive and process uploaded images.
- Database integration to persist image records.
- Cloud or object storage (for example, for storing the actual image files).
- User authentication to associate uploads with individual accounts.
- Drag-and-drop support as an alternative to the click-to-browse upload area.
- Support for multiple image uploads in a single submission.
- Client-side or server-side image compression before submission.
- Basic cropping tools within the preview panel.
- File-size validation to reject oversized images before they are processed.
- MIME-type validation to restrict selection to genuine image files.
- An upload progress indicator once a real backend is introduced.
- A user gallery view for previously submitted images.

12. Deployment and Maintenance

The project follows a standard Git-based workflow: source files are committed locally with Git and pushed to a public GitHub repository, which acts as the single source of truth for the project's history and version tracking.

The GitHub repository is connected to Vercel, which builds and serves the project as a static site. Because the application has no build step or backend dependency, deployment consists simply of publishing the HTML, CSS, and JavaScript files directly, giving a fast, reliable, publicly accessible live URL.

12.1 Updating the Project

When changes are made — for example, an adjustment to the CSS theme or a new validation rule in the JavaScript — the updated files are committed and pushed to GitHub. Vercel automatically detects the new commit and redeploys the static site, so the live version reflects the latest changes within moments, without any manual server configuration.

12.2 Testing After Deployment

After each deployment, the core flow — selecting an image, confirming the preview and filename update correctly, enlarging the preview, and submitting with and without an image selected — is re-tested on the live Vercel URL to confirm that the deployed build behaves identically to the local development version.

13. Conclusion

The Image Upload & Preview Web Application successfully demonstrates that a genuinely useful, well-designed image-selection experience can be built entirely on the frontend, using HTML5, CSS3, and vanilla JavaScript. The project meets every objective set for its scope: an instantly responsive preview, clear filename and validation feedback, an enlargeable preview state, a temporary success notification, and a modern, responsive dark/purple glassmorphism interface.

Developing the project reinforced practical, transferable skills in DOM manipulation, event-driven programming, asynchronous file reading with the FileReader API, conditional UI state management, and responsive CSS design — all without relying on a framework. It also provided direct experience with a complete deployment pipeline, from Git and GitHub through to a live static deployment on Vercel.

The clearly acknowledged boundary between this frontend-only prototype and a production-ready upload system — one with a backend, database, authentication, and real file storage — gives the project a natural and well-defined path for future development, as outlined in Section 11.3.